Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Σχεδίαση Ενσωματωμένων Συστημάτων

Εργαστηριακή Αναφορά: Βελτιστοποίηση Κώδικα και Εξερεύνηση Χώρου Σχεδίασης (DSE) του Αλγορίθμου PHODS στον Μικροελεγκτή Renesas S7G2

ΜΕΤΣΗΣ ΑΧΙΛΛΕΑΣ-ΟΔΥΣΣΕΑΣ - 03117403, ΙΑΣΩΝ-ΔΑΝΙΗΛ ΖΑΚΥΝΘΙΝΟΣ - 03123408

Νοέμβριος 2026

1 Ζητούμενο 1: Βελτιστοποίηση του PHODS στην πλακέτα S7G2

Abstract

Ο αλγόριθμος εκτίμησης κίνησης PHODS αποτελεί βασικό υπολογιστικό πυρήνα σε συστήματα συμπίεσης βίντεο, εντοπίζοντας τα διανύσματα κίνησης (motion vectors) μεταξύ διαδοχικών πλαισίων εικόνας (`current` και `previous`) διαστάσεων QCIF (144×176 εικονοστοιχεία). Λόγω των αυστηρών περιορισμών χρόνου πραγματικής εκτέλεσης (real-time constraints) και κατανάλωσης ενέργειας στα ενσωματωμένα συστήματα, η αποδοτική υλοποίηση του πυρήνα `phods_motion_estimation` είναι κρίσιμη.

Τα βασικά ερωτήματα της εργαστηριακής άσκησης που απαντώνται στην παρούσα αναφορά είναι:

1. **Τεχνικά χαρακτηριστικά** Μεγεθός μνημών του συστήματος και ποιά η λειτουργία τους καθώς και συχνότητα λειτουργίας
2. **Μέτρηση Χρόνου Εκτέλεσης (Profiling):** Ακριβής μέτρηση του χρόνου εκτέλεσης της συνάρτησης `phods_motion_estimation` στην πλακέτα Renesas S7G2 με χρήση του μετρητή κύκλων DWT και μετατροπή σε μικροδευτερόλεπτα (μs).
3. **Βελτιστοποίηση Κώδικα (Loop Optimizations):** Εφαρμογή μετασχηματισμών βρόχων (Loop Fusion, Loop-Invariant Code Motion) για τη μείωση του υπολογιστικού κόστους και τη βελτίωση της τοπικότητας μνήμης.
4. **Μονοδιάστατη Εξερεύνηση Χώρου Σχεδίασης (1D DSE):** Αποτίμηση της επίδρασης του μεγέθους τετραγωνικού μπλοκ $B \times B$ στον χρόνο εκτέλεσης και εύρεση της βέλτιστης τιμής B.
5. **Δισδιάστατη Εξερεύνηση Χώρου Σχεδίασης (2D DSE):** Γενίκευση του αλγορίθμου για ορθογώνια μπλοκ διαστάσεων $B_x \times B_y$ και εύρεση του βέλτιστου συνδυασμού (B_x, B_y).

1.1 Τεχνικά χαρακτηριστικά

Απο το datasheet προκύπτει οτι το μέγεθος της RAM είναι 640 KB χωρισμένη σε 512 KB βασικής μνήμης συστήματος και 128 KB δευτερεύουσας μνήμης για Error corection code και parity checking .Οι δύο flash memory είναι Code Flash Memory : 4 MB , Data Flash Memory 64 KB καθώς και η συχνότητατ του ρολογιού η οποία είναι 240 MHz,πληροφορία η οποία επιβεβαιώνεται και απο το e2studio.Τέλος υπάρχει μια ακόμα μνήμη Standby SRAM 8 KB η οποία διατηρεί τη κατάσταση του συστήματος σε Deep software standby mode οταν δεν παρέχεται ρεύμα στην SRAM.

1.2 Μεθοδολογία Μέτρησης Χρόνου Εκτέλεσης & Εξαγωγής Δεδομένων

Για την καταγραφή του ακριβούς αριθμού κύκλων ρολογιού (CPU cycles) στον πυρήνα ARM Cortex-M4 του μικροελεγκτή Renesas S7G2, χρησιμοποιήθηκε η μονάδα Data Watchpoint and Trace (DWT). Σε αντίθεση με τους εξωτερικούς χρονιστές (timers), ο καταχωρητής 32-bit `DWT->CYCCNT` αυξάνεται αυτόματα σε κάθε κύκλο ρολογιού του επεξεργαστή χωρίς να εισάγει καθυστέρηση διακοπών (interrupt overhead).

Η συχνότητα του ρολογιού συστήματος (`SystemCoreClock`) στον Renesas S7G2 ανέρχεται σε $f_{\text{CPU}} = 240$ MHz. Επομένως, ο χρόνος εκτέλεσης T_{exec} σε μικροδευτερόλεπτα (μs) υπολογίζεται από τη σχέση:

$$T_{\text{exec}}\ (\mu\text{s}) = \frac{N_{\text{cycles}}}{\text{SystemCoreClock}/10^6} = \frac{N_{\text{cycles}}}{240} \tag{1}$$

Παρακάτω παρουσιάζεται ο κώδικας αρχικοποίησης του καταχωρητή DWT, η εκτέλεση της μέτρησης για κάθε μέγεθος μπλοκ και η εξαγωγή των δεδομένων σε αρχείο CSV (μέσω semihosting ή πίνακα στη μνήμη RAM για ανάγνωση από τον debugger).

```c
#include "hal_data.h"
#include <stdio.h>

extern void initialise_monitor_handles(void); // Semihosting support

void run_dse_benchmark(void) {
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;

    uint32_t freq_mhz = SystemCoreClock / 1000000U; // 240 MHz
    FILE *fp = fopen("dse_results.csv", "w");
    if (fp != NULL) {
        fprintf(fp, "Block_B,Cycles,Time_us\n");
    }

    const int B_candidates[] = {1, 2, 4, 8, 10, 12, 16, 20, 32};
    int num_candidates = sizeof(B_candidates) / sizeof(B_candidates[0]);

    for (int idx = 0; idx < num_candidates; idx++) {
        int B = B_candidates[idx];

        /* 2. Initialize counter*/
        DWT->CYCCNT = 0;
        phods_motion_estimation(current, previous, vectors_x, vectors_y, B);
        uint32_t cycles = DWT->CYCCNT;

        /* 3.Cycles into us */
        double time_us = (double)cycles / (double)freq_mhz;

        if (fp != NULL) {
            fprintf(fp, "%d,%lu,%.2f\n", B, (unsigned long)cycles, time_us);
        }
    }

    if (fp != NULL) fclose(fp);
}
```

Listing 1: Μέτρηση κύκλων εκτέλεσης μέσω DWT->CYCCNT και εξαγωγή αποτελεσμάτων DSE σε CSV.

1.3 Βελτιστοποιήσεις Κώδικα (Loop Optimizations)

Στην αρχική (baseline) υλοποίηση της συνάρτησης `phods_motion_estimation`, εντοπίστηκαν σημαντικές πηγές υπολογιστικής επιβάρυνσης:

1. **Επαναλαμβανόμενοι υπολογισμοί δεικτών (Index Arithmetic):** Εκφράσεις της μορφής `B*x + k` και `B*y + l` υπολογίζονταν εκ νέου μέσα στον βαθύτερο βρόχο για κάθε εικονοστοιχείο, ενώ τα `vectors_x[x][y]` και `vectors_y[x][y]` διαβάζονταν από τη μνήμη σε κάθε επανάληψη.

2. **Έλεγχοι ορίων ανά εικονοστοιχείο:** Για κάθε εικονοστοιχείο εκτελούνταν τέσσερις συγκρίσεις (`if`) για το αν η θέση του `previous` βρίσκεται μέσα στο πλαίσιο, δηλαδή διακλαδώσεις μέσα στον εσωτερικότερο βρόχο.

3. **Διπλός υπολογισμός του κεντρικού υποψηφίου:** Σε κάθε βήμα S ο υποψήφιος $i = 0$ (θέση (vx, vy)) υπολογιζόταν δύο φορές, μία στην αναζήτηση κατά X και μία κατά Y, παρότι πρόκειται για το ίδιο ακριβώς block.

4. **Αναγνώσεις από τη Flash:** Ο πίνακας `current` είναι `const`, άρα βρίσκεται στη Flash, και κάθε εικονοστοιχείο του block διαβάζεται από εκεί σε κάθε SAD (3 βήματα $\times$ 6 SADs).

Για την ελαχιστοποίηση του χρόνου εκτέλεσης εφαρμόστηκαν σταδιακά οι εξής μετασχηματισμοί, κάθε ένας πάνω στον προηγούμενο (εκδόσεις `v1`–`v4` του Πίνακα 1):

- `v1` – **Loop-Invariant Code Motion & Scalar Replacement:** Τα `bx = B*x` και `by = B*y` υπολογίζονται μία φορά ανά block, τα διανύσματα κίνησης κρατούνται σε τοπικές μεταβλητές (`vx`, `vy`) και γράφονται μία φορά στο τέλος (οπότε αφαιρείται και ο αρχικός βρόχος μηδενισμού), ενώ ο έλεγχος ορίων γραμμής, που εξαρτάται μόνο από το `k`, μεταφέρεται έξω από τον βρόχο `l`.

- `v2` – **Εξάλειψη Ελέγχων Ορίων (Bounds-Check Elimination):** Ο υπολογισμός του SAD μεταφέρεται στη συνάρτηση `phods_sad`, η οποία υπολογίζει *μία φορά ανά υποψήφιο* το έγκυρο διάστημα γραμμών $[k_{\min}, k_{\max})$ και στηλών $[l_{\min}, l_{\max})$. Τα εικονοστοιχεία εκτός πλαισίου συνεισφέρουν $|p_1|$ (αφού $p_2 = 0$), ενώ για ολόκληρες γραμμές εκτός πλαισίου χρησιμοποιείται το προϋπολογισμένο άθροισμα `rowabs[k]`. Έτσι ο εσωτερικότερος βρόχος δεν έχει κανένα `if` και διατρέχει το `previous` με δείκτη (`*pp++`).

- `v3` – **Κοινός Κεντρικός Υποψήφιος:** Το SAD της θέσης (vx, vy) υπολογίζεται μία φορά ανά βήμα (`dc`) και χρησιμοποιείται και στις δύο αναζητήσεις, δηλαδή 5 SADs ανά βήμα αντί για 6.

- `v4` – **Αντιγραφή του Block από Flash σε RAM:** Το τρέχον block αντιγράφεται μία φορά ανά block σε τοπικό buffer `blk[]` στη στοίβα (SRAM), ώστε όλες οι επόμενες αναγνώσεις του να γίνονται από τη RAM με βήμα B αντί για M.

Σε όλες τις εκδόσεις τα διανύσματα κίνησης που προκύπτουν είναι ίδια με του αρχικού κώδικα (ίδια σειρά υποψηφίων και ίδιος χειρισμός ισοπαλιών). Στο Listing 2 παρατίθεται η τελική έκδοση (`phods_v4_copy`), η οποία περιλαμβάνει και τους τέσσερις μετασχηματισμούς.

```
/* SAD between the current block cb (row stride cs, per-row |.| sums in rowabs)
 * and the block of prev whose top-left corner is (r, c).                    */
static int phods_sad(const int *cb, int cs, const int *rowabs,
                     const int prev[N][M], int r, int c)
{
    int k, l, d, sum = 0;
    int kmin, kmax, lmin, lmax;
    const int *cp, *pp;

    /* v2: valid row / column range, computed once per candidate */
    kmin = (r < 0) ? -r : 0;  if (kmin > B) kmin = B;
    kmax = N - r;             if (kmax > B) kmax = B;  if (kmax < kmin) kmax = kmin
    ;
    lmin = (c < 0) ? -c : 0;  if (lmin > B) lmin = B;
    lmax = M - c;             if (lmax > B) lmax = B;  if (lmax < lmin) lmax = lmin
    ;
    if (lmin == lmax) kmax = kmin;      /* no column inside the frame */

```

```
    /* rows completely outside the frame: p2 = 0, use the precomputed row sum */
    for (k = 0; k < kmin; k++)     sum += rowabs[k];
    for (k = kmax; k < B; k++)     sum += rowabs[k];

    for (k = kmin; k < kmax; k++) {
        cp = cb + k*cs;
        pp = &prev[r+k][c+lmin];        /* first in-frame pixel of this row */
        /* no if in the inner loops */
        for (l = 0;    l < lmin; l++) { d = cp[l];          sum += (d < 0) ? -d : d;
    }
        for (l = lmin; l < lmax; l++) { d = cp[l] - *pp++; sum += (d < 0) ? -d : d;
    }
        for (l = lmax; l < B;    l++) { d = cp[l];          sum += (d < 0) ? -d : d;
    }
    }
    return sum;
}

void phods_v4_copy(const int cur[N][M], const int prev[N][M],
                   int vectors_x[N/B][M/B], int vectors_y[N/B][M/B])
{
    int x, y, k, l, S, d, dc, min1, min2, bestx, besty, vx, vy, r0, c0;
    int rowabs[B];
    int blk[B*B];                       /* v4: current block in RAM */

    for (x = 0; x < N/B; x++) {
        r0 = B*x;                       /* v1: LICM */
        for (y = 0; y < M/B; y++) {
            c0 = B*y;

            /* v4: copy the block from Flash (const) to RAM once per block */
            for (k = 0; k < B; k++)
                for (l = 0; l < B; l++)
                    blk[k*B + l] = cur[r0+k][c0+l];

            /* v2: |p1| sum of every row, for rows outside the frame */
            for (k = 0; k < B; k++) {
                rowabs[k] = 0;
                for (l = 0; l < B; l++) {
                    d = blk[k*B + l];
                    rowabs[k] += (d < 0) ? -d : d;
                }
            }

            vx = 0;                     /* v1: scalar replacement */
            vy = 0;
            for (S = 4; S > 0; S >>= 1) {
                min1 = 255*B*B;  min2 = 255*B*B;
                bestx = 0;       besty = 0;

                /* v3: centre candidate (vx, vy) computed once per step */
                dc = phods_sad(blk, B, rowabs, prev, r0+vx, c0+vy);

                /* X dimension: i = -S, 0, +S */
                d = phods_sad(blk, B, rowabs, prev, r0+vx-S, c0+vy);
                if (d < min1) { min1 = d; bestx = -S; }
                if (dc < min1) { min1 = dc; bestx = 0; }
                d = phods_sad(blk, B, rowabs, prev, r0+vx+S, c0+vy);
                if (d < min1) { min1 = d; bestx = S; }

                /* Y dimension: i = -S, 0, +S */
                d = phods_sad(blk, B, rowabs, prev, r0+vx, c0+vy-S);
                if (d < min2) { min2 = d; besty = -S; }
                if (dc < min2) { min2 = dc; besty = 0; }
                d = phods_sad(blk, B, rowabs, prev, r0+vx, c0+vy+S);
```

```
                if (d < min2) { min2 = d; besty = S; }

                vx += bestx;
                vy += besty;
            }

            vectors_x[x][y] = vx;       /* written once per block */
            vectors_y[x][y] = vy;
        }
    }
}
```

Listing 2: Τελική βελτιστοποιημένη υλοποίηση της `phods_motion_estimation` (`phods_v4_copy`): LICM, εξάλειψη ελέγχων ορίων, κοινός κεντρικός υποψήφιος και αντιγραφή του block σε RAM.

Ακολουθεί ο πίνακας με τα αποτελέσματα ανα βελτιστοποίησει που κάναμε στον κώδικα

Table 1: Impact of code optimizations on execution metrics ($N = 10, M = 10, B = 5$, 10 runs at 240 MHz).

Variant	Optimization Applied	Min Cyc.	Max Cyc.	Avg Cyc.	Min (μs)	Max (μs)	Avg (μs)	Speedup
`phods_v0_original`	Baseline (unmodified)	55139	55139	55139.00	229.75	229.75	229.75	1.00×
`phods_v1_hoist`	LICM + scalar replacement	40977	40977	40977.00	170.74	170.74	170.74	1.35×
`phods_v2_bounds`	+ Bounds-check elimination	33583	33584	33583.10	139.93	139.93	139.93	1.64×
`phods_v3_share`	+ Shared center SAD (vx, vy)	33654	33654	33654.00	140.23	140.23	140.23	1.64×
`phods_v4_copy`	+ Block copy Flash → RAM	**32670**	**32670**	**32670.00**	**136.13**	**136.13**	**136.13**	**1.69×**

Σχολιασμός Αποτελεσμάτων Πίνακα 1 (Βελτιστοποιήσεις Κώδικα)

Από τις μετρήσεις του Πίνακα 1 ($N = 10, M = 10, B = 5$, $f_{\text{CPU}} = 240$ MHz) προκύπτουν τα εξής συμπεράσματα σχετικά με την επίδραση κάθε μετασχηματισμού:

- **Αρχική Υλοποίηση (`phods_v0_original`):** Ο αρχικός κώδικας απαιτεί κατά μέσο όρο 55139.00 κύκλους ρολογιού (229.75 μs). Η υψηλή αυτή τιμή οφείλεται στον επαναλαμβανόμενο υπολογισμό των εκφράσεων $B \cdot x$ και $B \cdot y$ σε κάθε εικονοστοιχείο, στις συνεχείς προσπελάσεις μνήμης στους πίνακες `vectors_x[x][y]` και `vectors_y[x][y]` μέσα στον εσωτερικό βρόχο `l`, καθώς και στους τέσσερις εμφωλευμένους ελέγχους ορίων (`if`) ανά pixel.

- **Μετακίνηση Αμετάβλητου Κώδικα & Scalar Replacement (`phods_v1_hoist`):** Η εξαγωγή των $B \cdot x, B \cdot y$ έξω από τους εσωτερικούς βρόχους (Loop-Invariant Code Motion), η αντικατάσταση των αναγνώσεων των πινάκων `vectors_x/y` από τοπικούς καταχωρητές (`vx`, `vy`), η αφαίρεση του αρχικού διπλού βρόχου μηδενισμού και η μετακίνηση του ελέγχου ορίων γραμμής (`row`) στον εξωτερικό βρόχο `k` επιφέρουν τη μεγαλύτερη μεμονωμένη βελτίωση. Οι κύκλοι μειώνονται στους 40977.00 (170.74 μs), επιτυγχάνοντας μείωση κατά 25.68% και επιτάχυνση (speedup) 1.35×.

- **Εξάλειψη Ελέγχων Ορίων (`phods_v2_bounds`):** Με τον προ-υπολογισμό του έγκυρου διαστήματος γραμμών $[k_{\min}, k_{\max})$ και στηλών $[l_{\min}, l_{\max})$ μία φορά ανά υποψήφιο block, καθώς και του αθροίσματος απόλυτων τιμών ανά γραμμή (`rowabs[k]`), εξαλείφονται πλήρως οι εντολές διακλάδωσης (`if`) από τον εσωτερικότερο βρόχο. Αυτό αποτρέπει τα pipeline stalls στον επεξεργαστή ARM Cortex-M4 και επιτρέπει σειριακή σάρωση δεικτών (`*pp++`), μειώνοντας τους κύκλους κατά επιπλέον 18.04% σε σχέση με την `v1` (στους 33583.10 κύκλους ή 139.93 μs), δηλαδή συνολική μείωση 39.09% και speedup 1.64× ως προς την `v0`.

- **Κοινή Χρήση Κεντρικού Υποψηφίου (`phods_v3_share`):** Σε κάθε βήμα $S \in \{4, 2, 1\}$, ο υποψήφιος $i = 0$ αντιστοιχεί στην ίδια θέση (vx, vy) τόσο στην οριζόντια (X) όσο και στην κάθετη (Y) αναζήτηση. Ο υπολογισμός του μία φορά ανά βήμα (5 κλήσεις SAD αντί για 6) οδηγεί σε 33654.00 κύκλους (140.23 μs), παρουσιάζοντας οριακή αύξηση +0.21% (+71 κύκλοι) έναντι της `v2`. Αυτό εξηγείται από το γεγονός ότι για $B = 5$, το κόστος διατήρησης της ενδιάμεσης

μεταβλητής `dc` (register pressure / spilling στη στοίβα) και της ξεχωριστής κλήσης της `PHODS_SAD` εκτός της ροής του compiler αντισταθμίζει το κέρδος σε τόσο μικρό μέγεθος block.

- **Αντιγραφή Block από Flash σε SRAM (`phods_v4_copy`):** Επειδή ο πίνακας `current` είναι δηλωμένος ως `const int`, αποθηκεύεται στη μνήμη Flash (ROM) του μικροελεγκτή, η οποία στα 240 MHz εμφανίζει wait states κατά την ανάγνωση, σε αντίθεση με την τοπική SRAM που προσφέρει ταχύτερη προσπέλαση. Η αντιγραφή του τρέχοντος block 5×5 σε τοπικό buffer `blk[]` στη στοίβα (RAM) μία φορά ανά block —ώστε οι 15 αναγνώσεις SAD (3 βήματα$\times$5 SADs) να γίνονται από τη RAM με stride B_y αντί για M— ρίχνει τον χρόνο εκτέλεσης στο ελάχιστο των **32670.00 κύκλων** (**136.13** μs). Συνολικά επιτυγχάνεται μείωση κύκλων κατά **40.75%** και τελικό speedup **1.69**$\times$ σε σχέση με τον αρχικό κώδικα, ενώ η μεταβλητή `correct = 1` επιβεβαιώνει ότι τα διανύσματα κίνησης παραμένουν πανομοιότυπα.

1.4 Μονοδιάστατη Εξερεύνηση (1D DSE) για Τετραγωνικό Μπλοκ $B \times B$

Σκοπός της μονοδιάστατης εξερεύνησης είναι η εύρεση του μεγέθους μπλοκ B που ελαχιστοποιεί τον χρόνο εκτέλεσης της `phods_motion_estimation` στην πλακέτα Renesas S7G2. Εξετάστηκαν διάφορες τιμές του B και καταγράφηκαν οι κύκλοι ρολογιού και ο αντίστοιχος χρόνος σε μs, όπως φαίνεται στον Πίνακα **??** και στο Σχήμα 1.

Table 2: Impact of square block size B ($B_x = B_y = B$) on execution metrics ($N = 10, M = 10$, 10 runs at 240 MHz).

Configuration	Block ($B \times B$)	Blocks	Min Cyc.	Max Cyc.	Avg Cyc.	Min (μs)	Max (μs)	Avg (μs)
`phods_dse_1_1`	1×1	100	108112	108114	108112.20	450.47	450.48	450.47
`phods_dse_2_2`	2×2	25	75768	75768	75768.00	315.70	315.70	315.70
`phods_dse_5_5`	5×5	4	33656	33656	33656.00	140.23	140.23	140.23
`phods_dse_10_10`	10×10	1	**24861**	**24861**	**24861.00**	**103.59**	**103.59**	**103.59**

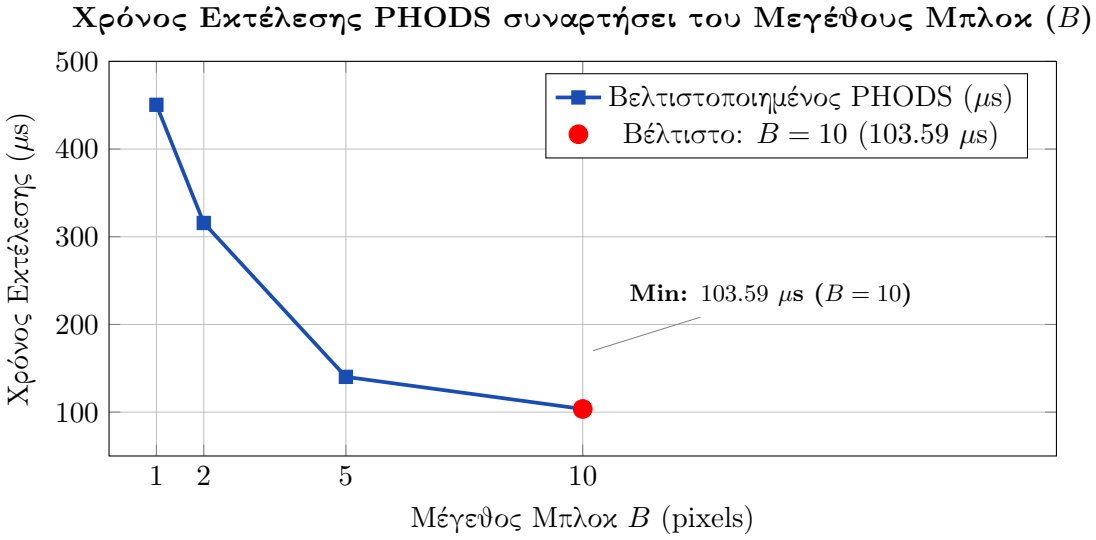


Figure 1: Καμπύλη εξερεύνησης χώρου σχεδίασης (DSE) για το μέγεθος μπλοκ B. Παρατηρείται η χαρακτηριστική μορφή «U» λόγω του trade-off μεταξύ loop overhead και τοπικότητας της cache.

Σχολιασμός Αποτελεσμάτων Πίνακα 2 (Επίδραση Τετραγωνικού Μεγέθους B)

Από τη διερεύνηση του μεγέθους τετραγωνικού block $B \in \{1, 2, 5, 10\}$ (όπου $B_x = B_y = B$) στον Πίνακα 2, παρατηρούμε ότι ο χρόνος εκτέλεσης μειώνεται μονοτονικά και δραστικά όσο αυξάνεται η διάσταση B:

- **Μείωση Κύκλων με την Αύξηση του B:** Η μετάβαση από $B = 1$ (108112.20 κύκλοι, 450.47 μs) σε $B = 2$ (75768.00 κύκλοι, 315.70 μs), $B = 5$ (33656.00 κύκλοι, 140.23 μs) και τελικά σε $B = 10$ (24861.00 κύκλοι, 103.59 μs) επιφέρει συνολική μείωση του χρόνου εκτέλεσης κατά **77.00%** (επιτάχυνση **4.35**× μεταξύ $B = 1$ και $B = 10$).

- **Αρχιτεκτονική Ερμηνεία (Amortization of Per-Block Overhead):** Παρότι το συνολικό πλήθος των εικονοστοιχείων του πλαισίου παραμένει σταθερό ($N \times M = 100$ pixels), το πλήθος των blocks ισούται με $(N/B) \times (M/B) = 100/B^2$. Για κάθε block εκτελούνται 3 βήματα αναζήτησης ($S = 4, 2, 1$) και 5 κλήσεις της συνάρτησης `PHODS_SAD` ανά βήμα (σύνολο 15 κλήσεις SAD ανά block). Κάθε κλήση της `PHODS_SAD` συνεπάγεται σταθερό κόστος (overhead) για τον υπολογισμό των ορίων $k_{\min}, k_{\max}, l_{\min}, l_{\max}$ και την αρχικοποίηση των βρόχων αναζήτησης.

  - Για $B = 1$ (100 blocks), εκτελούνται $100 \times 15 = 1500$ κλήσεις SAD για μόλις 1 pixel η καθεμία, με αποτέλεσμα το overhead ελέγχου ορίων και βρόχων να κυριαρχεί πλήρως στον χρόνο εκτέλεσης.
  - Για $B = 10$ (1 block), οι υπολογισμοί ορίων και η αναζήτηση (vx, vy) εκτελούνται μόνο για 1 block (15 κλήσεις SAD συνολικά), αποσβένοντας (amortizing) το loop και bounds-check overhead πάνω σε 100 pixels ταυτόχρονα.

- **Συμβιβασμός (Trade-off) Απόδοσης – Ακρίβειας Κίνησης:** Από καθαρά υπολογιστική σκοπιά, το $B = 10$ είναι το βέλτιστο (ελάχιστοι κύκλοι: 24861). Ωστόσο, σε πραγματική εκτίμηση κίνησης βίντεο, η επιλογή $B = 10$ σε εικόνα 10×10 αποδίδει μόλις ένα (1) διάνυσμα κίνησης για ολόκληρο το πλαίσιο, χάνοντας την τοπική κίνηση επιμέρους αντικειμένων. Αντίθετα, το $B = 5$ (33656 κύκλοι, 3.21× ταχύτερο από το $B = 1$) προσφέρει πολύ καλή ισορροπία μεταξύ ταχύτητας εκτέλεσης και χωρικής διακριτικής ικανότητας.

1.5 Δισδιάστατη Εξερεύνηση (2D DSE) για Ορθογώνιο Μπλοκ $B_x \times B_y$

Στο τελευταίο ερώτημα της άσκησης, ο αλγόριθμος τροποποιήθηκε ώστε να δέχεται ανεξάρτητες διαστάσεις μπλοκ B_x (κάθετο ύψος μπλοκ) και B_y (οριζόντιο πλάτος μπλοκ). Στο Listing 3 φαίνονται οι αλλαγές στους βρόχους για την υποστήριξη ορθογώνιων μπλοκ $B_x \times B_y$.

```
/* Internal loops on phods_motion_estimation_rect(..., int Bx, int By) */
for (x = 0; x < N / Bx; x++) {
    const int bx = Bx * x;
    for (y = 0; y < M / By; y++) {
        const int by = By * y;
        /* ... */
        for (k = 0; k < Bx; k++) {
            for (l = 0; l < By; l++) {
                /* Calculate SAD on grid Bx * By */
            }
        }
    }
}
```

Listing 3: Τροποποίηση των βρόχων για υποστήριξη ορθογώνιων μπλοκ $B_x \times B_y$ και αυτοματοποιημένο 2D DSE.

Table 3: Design space exploration for rectangular block sizes $B_x \times B_y$ ($B_x, B_y \in \{1, 2, 5, 10\}$).

Configuration	B_x	B_y	Blocks	Min Cyc.	Max Cyc.	Avg Cyc.	Min (μs)	Max (μs)	Avg (μs)
phods_dse_1_1	1	1	100	108112	108114	108112.20	450.47	450.48	450.47
phods_dse_1_2	1	2	50	80852	80852	80852.00	336.88	336.88	336.88
phods_dse_1_5	1	5	20	46575	46575	46575.00	194.06	194.06	194.06
phods_dse_1_10	1	10	10	33209	33210	33209.90	138.37	138.38	138.37
phods_dse_2_1	2	1	50	109150	109150	109150.00	454.79	454.79	454.79
phods_dse_2_2	2	2	25	75768	75768	75768.00	315.70	315.70	315.70
phods_dse_2_5	2	5	10	42029	42029	42029.00	175.12	175.12	175.12
phods_dse_2_10	2	10	5	31730	31730	31730.00	132.21	132.21	132.21
phods_dse_5_1	5	1	20	65440	65443	65440.30	272.67	272.68	272.67
phods_dse_5_2	5	2	10	48404	48405	48404.10	201.68	201.69	201.68
phods_dse_5_5	5	5	4	33656	33656	33656.00	140.23	140.23	140.23
phods_dse_5_10	5	10	2	26519	26521	26519.20	110.50	110.50	110.50
phods_dse_10_1	10	1	10	50555	50559	50555.40	210.65	210.66	210.65
phods_dse_10_2	10	2	5	39531	39531	39531.00	164.71	164.71	164.71
phods_dse_10_5	10	5	2	29955	29955	29955.00	124.81	124.81	124.81
phods_dse_10_10	10	10	1	**24861**	**24861**	**24861.00**	**103.59**	**103.59**	**103.59**

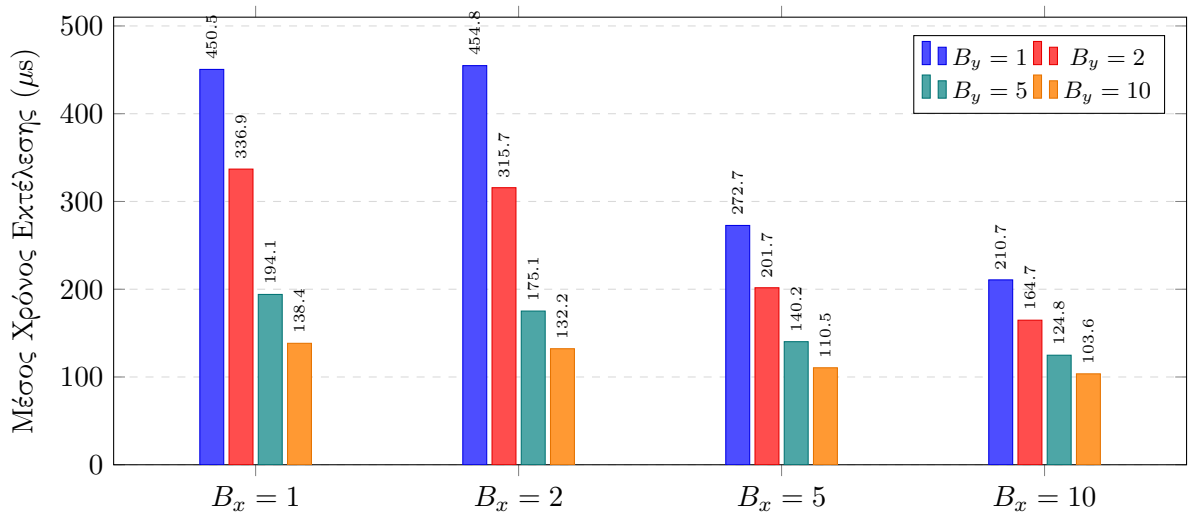


Figure 2: Σύγκριση μέσου χρόνου εκτέλεσης (μs) για όλους τους συνδυασμούς $B_x \times B_y$ ($B_x, B_y \in \{1, 2, 5, 10\}$).

Σχολιασμός Αποτελεσμάτων Πίνακα 3 (Εξερεύνηση Χώρου Σχεδίασης $B_x \times B_y$)

Η πλήρης εξερεύνηση χώρου σχεδίασης (Design Space Exploration – DSE) για όλους τους 16 συνδυασμούς $B_x, B_y \in \{1, 2, 5, 10\}$ στον Πίνακα 3 και στο αντίστοιχο διάγραμμα οδηγεί στα εξής ευρήματα:

- **Βέλτιστος και Χειρότερος Συνδυασμός:** Ο ταχύτερος συνδυασμός είναι ο $(B_x, B_y) = (10, 10)$ με **24861.00 κύκλους** (**103.59** μs), ενώ ο βραδύτερος είναι ο $(B_x, B_y) = (2, 1)$ με 109150.00 κύκλους (454.79 μs), ακολουθούμενος από τον $(1, 1)$ με 108112.20 κύκλους (450.47 μs). Αξίζει να σημειωθεί ότι ο $(2, 1)$ είναι ελαφρώς πιο αργός κατά 0.96% από τον $(1, 1)$ παρά το ότι έχει τα μισά blocks (50 έναντι 100), διότι με $B_y = 1$ ο εσωτερικός βρόχος στηλών εκτελείται για 1 μόνο στοιχείο ενώ ο εξωτερικός βρόχος γραμμών (`k`) της `PHODS_SAD` διπλασιάζει τις επαναλήψεις του.

- **Ασυμμετρία Απόδοσης μεταξύ B_x και B_y ($B_x \times B_y$ έναντι $B_y \times B_x$):** Το πιο ενδιαφέρον αρχιτεκτονικό εύρημα προκύπτει από τη σύγκριση συμμετρικών ζευγών (B_x, B_y) και (B_y, B_x), τα οποία έχουν *ακριβώς το ίδιο εμβαδόν block* και *το ίδιο πλήθος blocks*:

- $(1, 10)$ vs $(10, 1)$ [10 blocks]: 33209.90 έναντι 50555.40 κύκλων (**34.31**% λιγότεροι κύκλοι, 1.52× ταχύτερο).
- $(1, 5)$ vs $(5, 1)$ [20 blocks]: 46575.00 έναντι 65440.30 κύκλων (**28.83**% λιγότεροι κύκλοι, 1.41× ταχύτερο).
- $(1, 2)$ vs $(2, 1)$ [50 blocks]: 80852.00 έναντι 109150.00 κύκλων (**25.93**% λιγότεροι κύκλοι, 1.35× ταχύτερο).
- $(2, 10)$ vs $(10, 2)$ [5 blocks]: 31730.00 έναντι 39531.00 κύκλων (**19.73**% λιγότεροι κύκλοι, 1.25× ταχύτερο).
- $(2, 5)$ vs $(5, 2)$ [10 blocks]: 42029.00 έναντι 48404.10 κύκλων (**13.17**% λιγότεροι κύκλοι, 1.15× ταχύτερο).
- $(5, 10)$ vs $(10, 5)$ [2 blocks]: 26519.20 έναντι 29955.00 κύκλων (**11.47**% λιγότεροι κύκλοι, 1.13× ταχύτερο).

- **Αιτιολόγηση της Υπεροχής του Μεγάλου B_y (Row-Major Locality & Loop Overhead):** Σε κάθε ζεύγος ίσου εμβαδού, η διάταξη με $B_y > B_x$ («οριζόντιο/πλατύ» block) είναι συστηματικά ταχύτερη από τη διάταξη με $B_x > B_y$ («κάθετο/στενό» block). Αυτό οφείλεται σε δύο παράγοντες:

  1. *Δομή εμφωλευμένων βρόχων στη `PHODS_SAD`:* Ο εξωτερικός βρόχος `for(k=kmin; k<kmax; k++)` εκτελείται έως B_x φορές και σε κάθε επανάληψη υπολογίζει τους δείκτες αρχής γραμμής (`cp = cb + k*cs` και `pp = &prev[r+k][c+lmin]`) και αρχικοποιεί 3 εσωτερικούς βρόχους για το l. Συνεπώς, μικρότερο B_x σημαίνει πολύ λιγότερες επαναλήψεις του εξωτερικού βρόχου και λιγότερες αρχικοποιήσεις δεικτών.
  2. *Χωρική Τοπικότητα (Spatial Locality) στη Μνήμη:* Στη γλώσσα C οι δισδιάστατοι πίνακες αποθηκεύονται κατά γραμμές (row-major order). Μεγάλο B_y σημαίνει ότι ο εσωτερικός βρόχος `l` διαβάζει συνεχόμενες θέσεις μνήμης (`*pp++`) πάνω στην ίδια γραμμή με μοναδιαίο βήμα (stride 1), αξιοποιώντας ιδανικά το instruction/data prefetching της Flash του S7G2. Χαρακτηριστικά, ο συνδυασμός $(1, 10)$ με 10 blocks (33209.90 κύκλοι) είναι ακόμη πιο γρήγορος και από τον τετραγωνικό $(5, 5)$ που έχει μόλις 4 blocks (33656.00 κύκλοι)!

2 Ζητούμενο 2: Design Space Exploration με τον Προσομοιωτή gem5

Στο δεύτερο μέρος της άσκησης μελετάται η επίδραση της ιεραρχίας μνήμης (caches) και του loop unrolling στον χρόνο εκτέλεσης μιας εφαρμογής, με χρήση του προσομοιωτή αρχιτεκτονικής gem5. Στη συνέχεια γίνεται εξερεύνηση του χώρου σχεδίασης (Design Space Exploration – DSE) τόσο εξαντλητικά όσο και με γενετικό αλγόριθμο, ως προς δύο αντικρουόμενους στόχους: τον χρόνο εκτέλεσης και τη συνολική μνήμη cache.

2.1 Πειραματική Διάταξη

Οι προσομοιώσεις εκτελέστηκαν με το έτοιμο Docker image του μαθήματος (`aferikoglou/gem5built:v1.0`, gem5 23.0) και τα configuration scripts του `learning_gem5/part1`. Τα χαρακτηριστικά του προσομοιωμένου συστήματος συνοψίζονται στον Πίνακα 4.

Table 4: Simulated system configuration (gem5, syscall emulation mode).

Parameter	**Value**
CPU model	`X86TimingSimpleCPU` @ 1 GHz
Main memory	DDR3-1600 (`DDR3_1600_8x8`)
L1I / L1D	2-way set associative, latency 2 cycles
L2	8-way set associative, latency 20 cycles
Cache line	64 bytes
Benchmark	`tables.c`, compiled with `gcc -O0`

Η εφαρμογή `tables.c` δεσμεύει τέσσερις πίνακες `double` των $N = 100000$ στοιχείων (800 KB ο καθένας, 3.2 MB συνολικά). Σε έναν πρώτο βρόχο τους αρχικοποιεί και σε έναν δεύτερο υπολογίζει `y[i] += a1*x1[i] + a2*x2[i] + a3*x3[i]`. Πρόκειται δηλαδή για καθαρό streaming μοτίβο προσβάσεων: κάθε στοιχείο χρησιμοποιείται μία φορά ανά βρόχο, χωρίς επαναχρησιμοποίηση (temporal locality).

2.2 Αρχιτεκτονική χωρίς Caches (`simple.py`)

Στο `simple.py` ο επεξεργαστής συνδέεται απευθείας στο memory bus, χωρίς καμία cache. Το script τροποποιήθηκε ώστε να δέχεται το εκτελέσιμο ως όρισμα (`sys.argv[1]`), όπως ζητά η εκφώνηση. Από το αρχείο `m5out/stats.txt` προκύπτει:

`system.cpu.numCycles` = **857689669** κύκλοι

2.3 Αρχιτεκτονική με Ιεραρχία Caches (`two_level.py`)

Με την προσθήκη δύο επιπέδων cache (L1I = 8 KB, L1D = 8 KB, L2 = 128 KB) στο `two_level.py`, ο αριθμός των κύκλων μειώνεται δραστικά, όπως φαίνεται στον Πίνακα 5.

Table 5: Execution cycles without and with a cache hierarchy (`tables.exe`).

System	**L1I**	**L1D**	**L2**	**numCycles**	**Speedup**
`simple.py` (no caches)	–	–	–	857689669	1.00×
`two_level.py`	8 KB	8 KB	128 KB	59787087	**14.35**×

Σχολιασμός Αποτελεσμάτων Πίνακα 5 (Επίδραση των Caches)

- **Χωρίς Caches:** Κάθε ανάκληση εντολής (instruction fetch) και κάθε load/store διασχίζει το memory bus μέχρι τη DRAM, με καθυστέρηση δεκάδων κύκλων. Το πρόγραμμα εκτελεί

περίπου 8.2 εκατομμύρια εντολές, με 11.6 εκατομμύρια ανακλήσεις εντολών και 4.4 εκατομμύρια προσβάσεις δεδομένων, οπότε ο χρόνος εκτέλεσης κυριαρχείται πλήρως από την καθυστέρηση της κύριας μνήμης.

- **Με Caches – Εντολές:** Ο κώδικας αποτελείται από λίγους, μικρούς βρόχους και χωράει άνετα στην L1I, η οποία παρουσιάζει miss rate μόλις 0.016% (temporal locality των εντολών). Σχεδόν όλες οι ανακλήσεις εντολών εξυπηρετούνται σε 2 κύκλους.
- **Με Caches – Δεδομένα:** Οι προσβάσεις δεδομένων εξυπηρετούνται κυρίως από την L1D ή την L2, χάρη στη χωρική τοπικότητα (spatial locality): μία γραμμή των 64 bytes περιέχει 8 διαδοχικά `double`. Ακόμη και όταν η L1D αστοχεί, τα δεδομένα βρίσκονται συχνά στην L2 (L2 miss rate 12.6%) και κοστίζουν περίπου 20 κύκλους αντί για μία πρόσβαση στη DRAM. Το συνολικό αποτέλεσμα είναι επιτάχυνση **14.35**×.

2.4 Επίδραση του Loop Unrolling και του Μεγέθους της L1D

2.4.1 Loop Unrolling

Με την ίδια αρχιτεκτονική (8/8/128 KB) προσομοιώθηκαν οι εκδόσεις `tables_uf{2,4,8,16,32}.exe`, όπου ο βρόχος υπολογισμού έχει ξεδιπλωθεί (unrolled) με τον αντίστοιχο παράγοντα. Τα αποτελέσματα φαίνονται στον Πίνακα 6 και στο Σχήμα 3.

Table 6: Execution cycles per unrolling factor (L1I = L1D = 8 KB, L2 = 128 KB).

Unrolling Factor	numCycles	Reduction vs UF=1	Speedup
1 (`tables.exe`)	59787087	–	1.000×
2	59734252	0.09%	1.001×
4	59489841	0.50%	1.005×
8	59340239	0.75%	1.008×
16	59296471	0.82%	1.008×
32	59391341	0.66%	1.007×

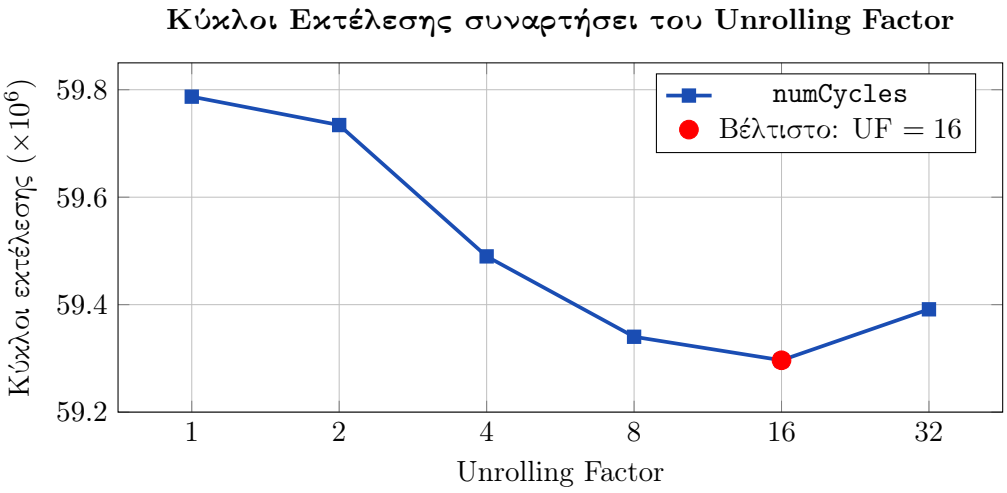


Figure 3: Κύκλοι εκτέλεσης για τα διάφορα unrolling factors. Η κλίμακα του άξονα y είναι στενή: η συνολική διαφορά είναι κάτω από 1%.

Σχολιασμός Αποτελεσμάτων Πίνακα 6 (Loop Unrolling)

- **Μικρό αλλά Υπαρκτό Όφελος:** Το unrolling μειώνει τους κύκλους μονοτονικά έως UF = 16, όπου επιτυγχάνεται το ελάχιστο (59296471 κύκλοι), αλλά το συνολικό κέρδος είναι μόλις

0.82%. Το όφελος προέρχεται από τη μείωση του loop overhead: με `-O0` ο μετρητής `i` βρίσκεται στη στοίβα, οπότε σε κάθε επανάληψη εκτελούνται load, αύξηση, store, σύγκριση και διακλάδωση. Με UF = 16 οι προσβάσεις στην L1D μειώνονται από 4.44 σε 4.06 εκατομμύρια.

- **Γιατί το Κέρδος είναι Μικρό:** Ο χρόνος εκτέλεσης δεν καθορίζεται από τις εντολές του βρόχου αλλά από τα L1D misses (περίπου 817 χιλιάδες με L1D = 8 KB), τα οποία το unrolling δεν επηρεάζει, αφού δεν αλλάζει τη σειρά ούτε το πλήθος των προσβάσεων στους πίνακες.
- **Επιδείνωση για UF = 32:** Για UF = 32 οι κύκλοι αυξάνονται ξανά. Το σώμα του βρόχου μεγαλώνει σημαντικά (ο κώδικας του `tables_uf32.exe` είναι 7.3 KB έναντι 1.9 KB του `tables.exe`), επιβαρύνοντας την L1I, ενώ δεν απομένει πλέον σημαντικό loop overhead για να εξοικονομηθεί. Η επίδραση αυτή γίνεται πολύ πιο έντονη με μικρότερη L1I (ενότητα 2.5).

2.4.2 Μέγεθος της L1D Cache

Στη συνέχεια, με το `tables.exe`, L1I = 8 KB και L2 = 128 KB, μεταβλήθηκε το μέγεθος της L1D από 2 έως 64 KB (Πίνακας 7, Σχήμα 4).

Table 7: Execution cycles and L1D statistics per L1D size (`tables.exe`, L1I = 8 KB, L2 = 128 KB).

L1D Size	numCycles	L1D Misses	L1D Miss Rate	Speedup vs 8 KB
2 KB	59973394	824278	18.6%	0.997×
4 KB	59824719	819107	18.5%	0.999×
8 KB	59787087	816756	18.4%	1.000×
16 KB	42811763	110454	2.5%	1.397×
32 KB	42580473	102101	2.3%	**1.404**×
64 KB	42842142	101993	2.3%	1.396×

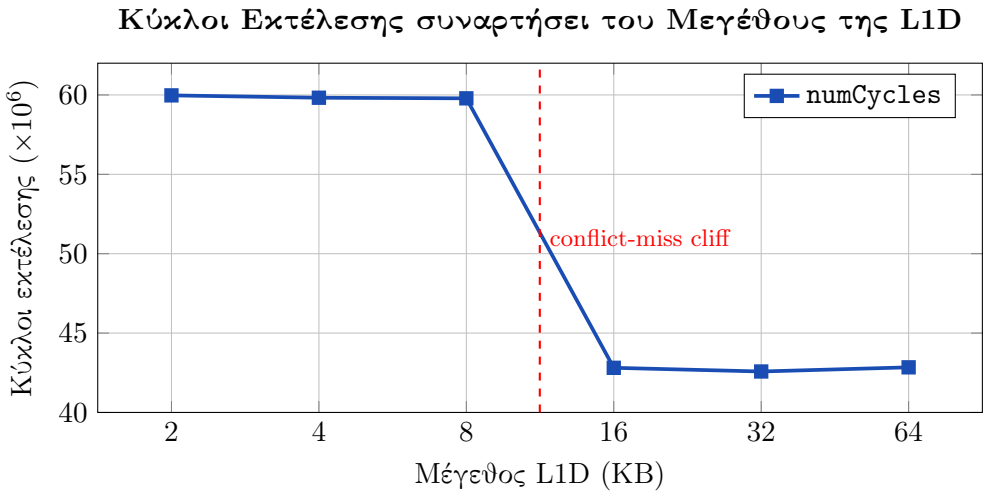


Figure 4: Κύκλοι εκτέλεσης συναρτήσει του μεγέθους της L1D. Παρατηρείται απότομη πτώση (−28%) μεταξύ 8 και 16 KB.

Σχολιασμός Αποτελεσμάτων Πίνακα 7 (Μέγεθος L1D)

Η καμπύλη δεν μειώνεται σταδιακά, όπως θα περίμενε κανείς από την αύξηση της χωρητικότητας. Μένει σχεδόν σταθερή έως τα 8 KB, πέφτει απότομα κατά 28% στα 16 KB και σταθεροποιείται ξανά. Η εξήγηση *δεν* είναι η χωρητικότητα, αφού τα δεδομένα (3.2 MB) είναι πολύ μεγαλύτερα από οποιαδήποτε από αυτές τις caches, αλλά τα **conflict misses**:

- **Διάταξη των Πινάκων στη Μνήμη:** Σε syscall emulation mode το gem5 δίνει φυσικές σελίδες κατά την πρώτη πρόσβαση (first touch). Ο βρόχος αρχικοποίησης αγγίζει εναλλάξ τα

`x1[i]`, `x2[i]`, `x3[i]` και `y[i]`, οπότε οι σελίδες των τεσσάρων πινάκων καταλήγουν διαδοχικές στη φυσική μνήμη, σε απόσταση 4 KB η μία από την άλλη.

- **Αντιστοίχιση σε Sets:** Σε μια 2-way cache μεγέθους C κάθε way καλύπτει $C/2$ bytes, και διευθύνσεις που διαφέρουν κατά πολλαπλάσιο του $C/2$ αντιστοιχίζονται στο ίδιο set.
  - Για L1D $\leq$ 8 KB το way είναι το πολύ 4 KB, άρα *και οι τέσσερις* πίνακες πέφτουν στο ίδιο set. Κάθε επανάληψη χρειάζεται 4 γραμμές σε ένα set με 2 θέσεις, οπότε κάθε πρόσβαση διώχνει μια γραμμή που θα χρειαστεί αμέσως μετά (thrashing): περίπου ένα miss ανά πρόσβαση ($\approx$ 817 χιλιάδες misses).
  - Για L1D = 16 KB το way είναι 8 KB: τα `x1`, `x3` μοιράζονται ένα set και τα `x2`, `y` ένα άλλο, οπότε οι δύο γραμμές κάθε set χωράνε. Τα misses πέφτουν στις $\approx$ 110 χιλιάδες, πολύ κοντά στο ελάχιστο των compulsory misses (3.2 MB/64 B = 50000 γραμμές σε κάθε έναν από τους δύο βρόχους).
- **Πάνω από 16 KB:** Δεν υπάρχει περαιτέρω κέρδος, αφού δεν υπάρχει επαναχρησιμοποίηση πέρα από τη γραμμή cache. Για αυτό το μοτίβο προσβάσεων λοιπόν η associativity και η στοίχιση των δεδομένων μετρούν περισσότερο από το καθαρό μέγεθος της cache.

2.5 Εξαντλητική Εξερεύνηση Χώρου Σχεδίασης (Exhaustive DSE)

Ο χώρος σχεδίασης ορίζεται από τις παραμέτρους του Πίνακα 8, δηλαδή $6 \times 6 \times 4 \times 5 = 720$ αρχιτεκτονικές. Όλες οι προσομοιώσεις εκτελέστηκαν αυτοματοποιημένα με Python script: κάθε προσομοίωση γράφει σε δικό της output directory (`-outdir`), ώστε να εκτελούνται 10 παράλληλα, και τα αποτελέσματα (`numCycles`, συνολική μνήμη) αποθηκεύονται σε αρχείο CSV.

Table 8: Design space of the exhaustive exploration (720 configurations).

Parameter	**Values**
L1I size	2, 4, 8, 16, 32, 64 KB
L1D size	2, 4, 8, 16, 32, 64 KB
L2 size	128, 256, 512, 1024 KB
Unrolling factor	2, 4, 8, 16, 32

Χρόνος εξαντλητικής αναζήτησης: οι 720 προσομοιώσεις ολοκληρώθηκαν σε 688 s με 10 παράλληλες εκτελέσεις. Το άθροισμα των επιμέρους χρόνων, δηλαδή ο χρόνος μιας σειριακής εξαντλητικής αναζήτησης, είναι **114 λεπτά** (περίπου 9.5 s ανά προσομοίωση). Η καλύτερη καθυστέρηση είναι 41922998 κύκλοι (L1I 16 / L1D 64 / L2 512 KB, UF = 16) και η χειρότερη 66060573 κύκλοι (L1I 2 / L1D 2 / L2 128 KB, UF = 32).

Το **Pareto front** υπολογίστηκε με τη βιβλιοθήκη `paretoset` πάνω σε `pandas` dataframe με στήλες `totalMemoryKB` (L1I + L1D + L2) και `latencyCC`, ελαχιστοποιώντας και τις δύο. Αποτελείται από 20 σημεία (Πίνακας 9, Σχήμα 5).

Table 9: Pareto-optimal architectures of the exhaustive search (minimize total memory and latency). The knee of the front is highlighted.

#	L1I	L1D	L2	UF	Total Memory (KB)	Latency (cycles)
1	2 KB	2 KB	128 KB	8	132	59581198
2	2 KB	4 KB	128 KB	8	134	59433698
3	4 KB	4 KB	128 KB	16	136	59343511
4	4 KB	8 KB	128 KB	16	140	59306017
5	8 KB	8 KB	128 KB	16	144	59296471
6	**2 KB**	**16 KB**	**128 KB**	**8**	**146**	**42416612**
7	4 KB	16 KB	128 KB	16	148	42409832
8	8 KB	16 KB	128 KB	16	152	42403308
9	16 KB	16 KB	128 KB	16	160	42387518
10	2 KB	32 KB	128 KB	8	162	42078234
11	4 KB	32 KB	128 KB	16	164	41969420
12	8 KB	32 KB	128 KB	16	168	41969226
13	16 KB	32 KB	128 KB	16	176	41945801
14	32 KB	32 KB	128 KB	16	192	41936488
15	16 KB	32 KB	256 KB	16	304	41933039
16	64 KB	32 KB	256 KB	16	352	41928359
17	64 KB	64 KB	256 KB	16	384	41925024
18	16 KB	32 KB	512 KB	16	560	41923846
19	32 KB	32 KB	512 KB	16	576	41923699
20	16 KB	64 KB	512 KB	16	592	41922998

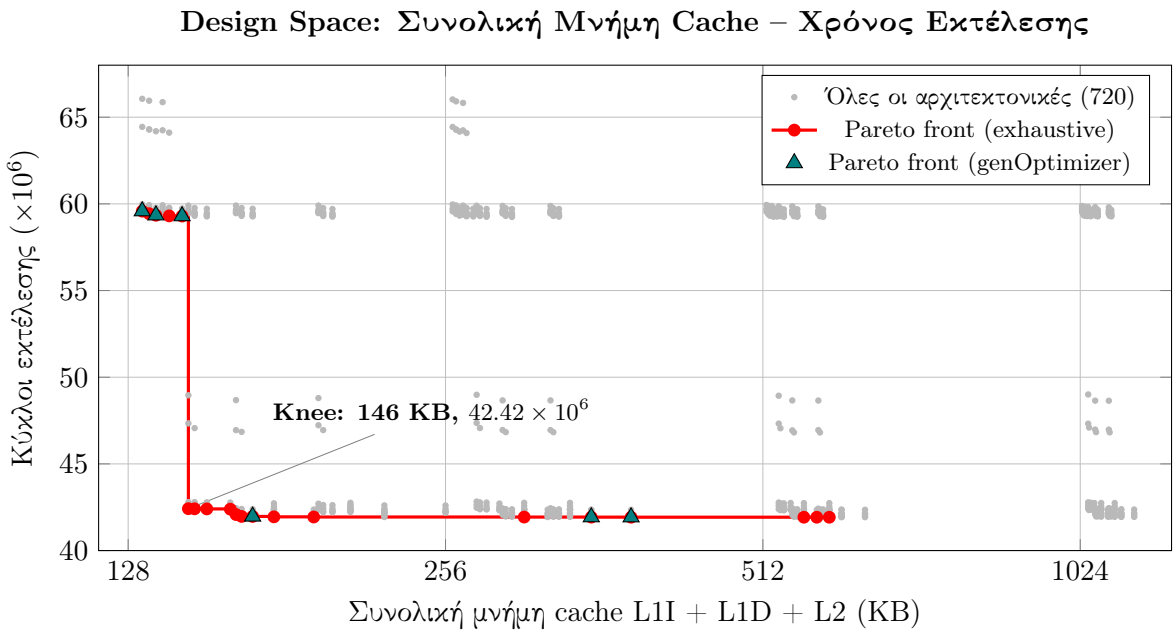


Figure 5: Οι 720 αρχιτεκτονικές της εξαντλητικής αναζήτησης (γκρι), το Pareto front τους (κόκκινο) και το Pareto front του γενετικού αλγορίθμου (πράσινα τρίγωνα). Ο άξονας x είναι σε λογαριθμική κλίμακα.

Σχολιασμός Αποτελεσμάτων Πίνακα 9 (Pareto Front)

- **Η L1D Καθορίζει την Απόδοση:** Το front χωρίζεται σε δύο σαφείς ομάδες. Οι αρχιτεκτονικές με L1D $\leq$ 8 KB βρίσκονται γύρω στους 59.3 εκατομμύρια κύκλους, ενώ όλες με L1D $\geq$ 16 KB πέφτουν κάτω από τους 42.5 εκατομμύρια. Η αποφυγή των conflict misses του προηγούμενου ερωτήματος μειώνει την καθυστέρηση κατά $\approx$ 29%, με κόστος μόλις 2–14 KB επιπλέον μνήμης.
- **Το «Γόνατο» του Front:** Η αρχιτεκτονική **L1I** 2 **KB** / **L1D** 16 **KB** / **L2** 128 **KB** με **UF** = 8 (146 KB, 42416612 κύκλοι) είναι η πιο αποδοτική επιλογή: από εκεί και πέρα, κάθε επιπλέον μνήμη (έως και 4× περισσότερη) κερδίζει λιγότερο από 1.2% σε χρόνο εκτέλεσης.

- **Η L2 Σχεδόν δεν Επηρεάζει:** Η καλύτερη καθυστέρηση για κάθε μέγεθος L2 διαφέρει λιγότερο από 0.05%. Τα δεδομένα διαβάζονται μία φορά (streaming), οπότε μια μεγαλύτερη L2 δεν έχει τι να επαναχρησιμοποιήσει. Γι' αυτό τα 14 από τα 20 Pareto σημεία έχουν τη μικρότερη L2 (128 KB).

- **Αλληλεπίδραση Unrolling και L1I:** Ένας βρόχος με μεγάλο unrolling δεν χωράει σε μικρή L1I (Πίνακας 10). Με L1I = 2 KB το βέλτιστο UF είναι το 8, ενώ με L1I $\geq$ 4 KB είναι το 16. Αυτό εξηγεί γιατί τα Pareto σημεία με L1I = 2 KB έχουν UF = 8 και όλα τα υπόλοιπα UF = 16.

Table 10: Latency ($\times 10^6$ cycles) per L1I size and unrolling factor (L1D = 32 KB, L2 = 128 KB).

L1I \ UF	**2**	**4**	**8**	**16**	**32**
2 KB	42.45	42.24	**42.08**	46.95	48.68
4 KB	42.40	42.19	42.03	**41.97**	46.85
$\geq$ 8 KB	42.39	42.18	42.01	**41.97**	42.16

2.6 Εξερεύνηση με Γενετικό Αλγόριθμο (genOptimizer)

Το `genOptimizer.py` του μαθήματος εκτελέστηκε χωρίς αλλαγές. Χρησιμοποιεί τον αλγόριθμο NSGA-II της βιβλιοθήκης `pymoo`, με πληθυσμό 10 και 5 απογόνους ανά γενιά, και τερματίζει μετά από 5 γενιές (`n_max_gen = 5`). Ο αλγόριθμος έκανε **30 αξιολογήσεις** ($10 + 4 \times 5$) σε **215 s**. Το Pareto front που επέστρεψε φαίνεται στον Πίνακα 11 και στο Σχήμα 5.

Table 11: Pareto front returned by the genetic algorithm and comparison with the true (exhaustive) front.

L1I	**L1D**	**L2**	**UF**	**Total Memory (KB)**	**Latency (cycles)**	**On true front?**
2 KB	2 KB	128 KB	8	132	59581198	✓
4 KB	4 KB	128 KB	16	136	59343511	✓
8 KB	8 KB	128 KB	16	144	59296471	✓
8 KB	32 KB	128 KB	16	168	41969226	✓
32 KB	64 KB	256 KB	16	352	41929862	×
64 KB	64 KB	256 KB	16	384	41925024	✓

Table 12: Cost comparison between exhaustive search and the genetic algorithm.

Method	**Simulations**	**Time**	**Pareto points found**
Exhaustive (sequential)	720	114 min	20 / 20
Exhaustive (10 parallel)	720	688 s	20 / 20
genOptimizer (NSGA-II)	30	215 s	5 / 20

Σχολιασμός Αποτελεσμάτων Πινάκων 11 και 12 (Γενετικός Αλγόριθμος έναντι Εξαντλητικής Αναζήτησης)

- **Κόστος Αναζήτησης:** Ο γενετικός αλγόριθμος χρειάστηκε μόλις το 4.2% των προσομοιώσεων (30 έναντι 720) και περίπου **32**× λιγότερο χρόνο από τη σειριακή εξαντλητική αναζήτηση (215 s έναντι 114 λεπτών).

- **Ποιότητα των Λύσεων:** Τα 5 από τα 6 σημεία που επέστρεψε είναι πράγματι Pareto-optimal. Το έκτο (32/64/256 KB) κυριαρχείται οριακά από την αρχιτεκτονική 64/32/256 KB, που έχει την ίδια συνολική μνήμη (352 KB) και 1503 κύκλους λιγότερους. Ωστόσο, ο αλγόριθμος βρήκε μόνο 5 από τα 20 σημεία του πραγματικού front.

- **Απώλεια του «Γονάτου»:** Το σημαντικότερο μειονέκτημα είναι ότι κανένα σημείο του γενετικού αλγορίθμου δεν έχει L1D = 16 KB. Έτσι, μεταξύ 144 και 168 KB το front του δίνει ακόμη 59.3 εκατομμύρια κύκλους, ενώ με 146 KB επιτυγχάνονται ήδη 42.4 εκατομμύρια, δηλαδή ακριβώς η πιο ενδιαφέρουσα σχεδιαστική επιλογή λείπει.

- **Συμπέρασμα:** Ο γενετικός αλγόριθμος δίνει τη γενική μορφή του trade-off (μικρή και αργή αρχιτεκτονική απέναντι σε μεγάλη και γρήγορη) με ένα κλάσμα του κόστους, αλλά με 5 γενιές και πληθυσμό 10 δεν προλαβαίνει να συγκλίνει. Περισσότερες γενιές ή μεγαλύτερος πληθυσμός θα έκλειναν τη διαφορά, με κόστος περισσότερες προσομοιώσεις. Εδώ η εξαντλητική αναζήτηση ήταν εφικτή μόνο επειδή κάθε προσομοίωση διαρκεί περίπου 9 s· σε μεγαλύτερο χώρο σχεδίασης ή με πιο αργές προσομοιώσεις, η ευρετική αναζήτηση είναι η μόνη ρεαλιστική επιλογή.